

Typed Intermediate Languages

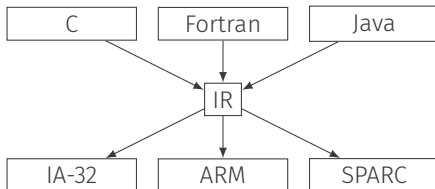
Roland Leißa

University of Göttingen

Introduction

Intermediate Representations (IRs)

- Assume we want to support k source languages and n targets
- Instead of writing $k \cdot n$ code generators
- Write $k + n$ by translating into a common IR and generate code from there
- Pays off because optimizations can be (mostly) source- and target-independent
- In practice: source and target influence the IR
- Popular IRs: Java Bytecode, .NET CIL, LLVM Bitcode, WebAssembly
- First proposal: UNCOL, UNiversal Computer Oriented Language (Melvin E. Conway, 1958)
- Functional languages: λ -calculus-based



Control-Flow Graphs

```
x = y + z * c + 5;
```

⇓

```
t1 = z * c;  
t2 = y + t1;  
x  = t2 + 5;
```

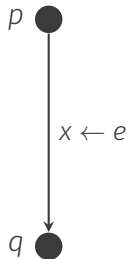
Control Flow Graph (CFG)

- Each instruction in a function has a set of possible successor instructions in that function
- **Control flow graph (CFG)** is a directed graph (V, E, r) , $E \subseteq V^2, r \in V$:
 - V : instructions of the function
 - $(v, w) \in E$ iff w is a possible successor instruction of v
 - r : A root node which has no predecessors
- Need not be connected. **Unreachable code**: nodes not reachable from r
- Typically IRs are **intra-procedural**:
 - Each function/procedure is given by a **control flow graph**
 - Is a design choice
- Control flow between functions (call, return, exceptions, etc.) is not modelled explicitly in the CFG
 - In general undecidable what the contents of a function pointer are
 - Call/Return behavior follows stack principle (cannot express in CFG)

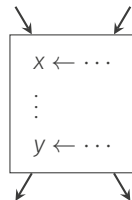
Basic Blocks

- It is common to summarize “straight-line” code segments of the CFG as **basic blocks**
- IRs typically use basic blocks as CFG nodes
- A basic block is a list of instructions with the following properties
 - the first instruction has multiple predecessors
 - the last instruction has multiple successors
 - every other instruction has one predecessor and one successor
- Reasons:
 - In many optimizations instructions with same control condition can be treated similarly (e.g. code motion, loop transformation, unreachable code elimination)
 - For some optimizations simpler special cases exist for straight-line code

Kinds of CFGs

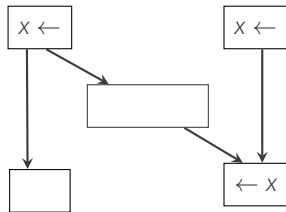
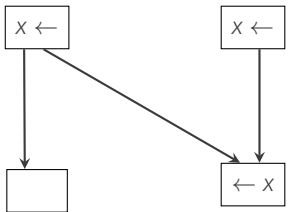


- Instructions on edges
- Nodes called program points
- Used in formal contexts (program analysis)



- Nodes are basic blocks
- Edges only denote control flow
- Close to actual implementation (BB = data structure, edge = pointer)

Critical Edges



- A CFG edge that goes from a block A with multiple successors to a block B with multiple predecessors is called **critical**
- Sometimes, the compiler needs to put code that is only executed if the program jumps from A to B
- To enable this, critical edges must be “broken”
- Loop back edges of **do** ... **while** loops and inverted loops are typically critical

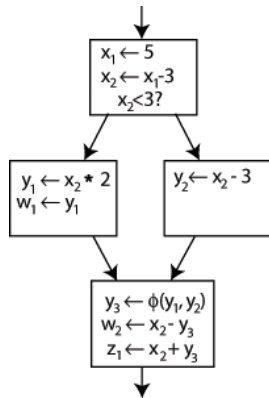
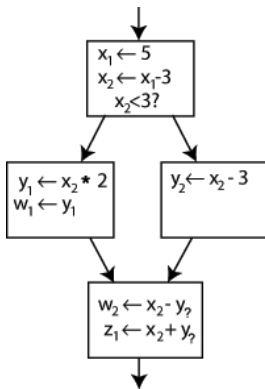
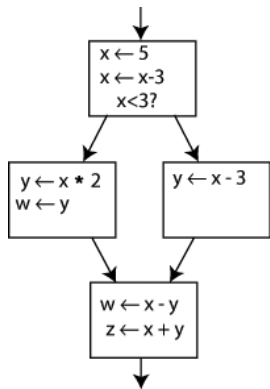
Static Single Assignment Form (SSA Form)

```
y := 1  
y := 2  
x := y
```

VS.

```
y1 := 1  
y2 := 2  
x1 := y2
```

ϕ -Functions



LLVM

- open source
- large, active research community
- used in industry:
 - Apple, Google, Intel, NVIDIA, Sony, ...
- front-ends for many languages:
 - C/C++, Fortran, Rust, Swift, Julia, Haskell, ...
- back-ends for many architectures:
 - X86(-64), ARM, RISC-V, MIPS, WebAssembly, ...

. HUGE



- open source
- large, active research community
- used in industry:
 - Apple, Google, Intel, NVIDIA, Sony, ...
- front-ends for many languages:
 - C/C++, Fortran, Rust, Swift, Julia, Haskell, ...
- back-ends for many architectures:
 - X86(-64), ARM, RISC-V, MIPS, WebAssembly, ...



. HUGE

- open source
- large, active research community
- used in industry:
Apple, Google, Intel, NVIDIA, Sony, ...
- front-ends for many languages:
C/C++, Fortran, Rust, Swift, Julia, Haskell, ...
- back-ends for many architectures:
X86(-64), ARM, RISC-V, MIPS, WebAssembly, ...



• HUGE

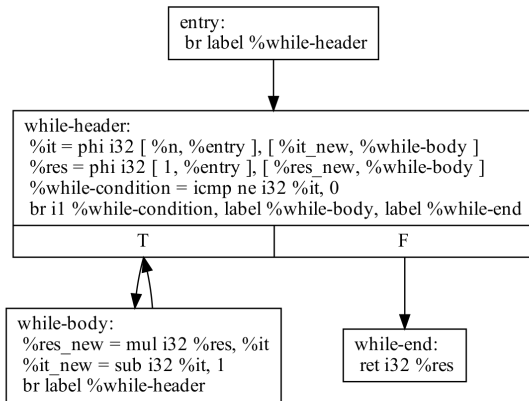
- SSA-based representation of control flow graphs
- dumpable in human-readable, assembly-like form (`*.ll`)
- dumpable as compact bitcode (`*.bc`)

LLVM Intermediate Representation — Example

```
define i32 @fac(i32 %n) {  
entry:  
  br label %while-header  
  
while-header:  
  %it = phi i32 [ %n, %entry ], [ %it_new, %while-body ]  
  %res = phi i32 [ 1, %entry ], [ %res_new, %while-body ]  
  %while-condition = icmp ne i32 %it, 0  
  br i1 %while-condition, label %while-body, label %while-end  
  
while-body:  
  %res_new = mul i32 %res, %it  
  %it_new = sub i32 %it, 1  
  br label %while-header  
  
while-end:  
  ret i32 %res  
}
```

```
int fac(int n) {  
  int it = n;  
  int res = 1;  
  
  while (it != 0) {  
  
    res = res * it;  
    it = it - 1;  
  }  
  
  return res;  
}
```

LLVM Intermediate Representation — Example



CFG for 'fac' function

Useful Commands

- Generate (human readable) LLVM-IR from C/C++ input:
`clang -Xclang -disable-00-optnone -emit-llvm -c -S -o OUT.ll IN.c`
- Draw CFG of function `foo` from dumped LLVM-IR module:
`opt -dot-cfg IN.ll; dot -Tpdf cfg.foo.dot > OUT.pdf` requires `dot/graphviz`
- Execute dumped LLVM-IR module:
`lli IN.ll <argv arguments>`
- Create binary from dumped LLVM-IR module:
`clang -o OUT IN.ll`
- Create architecture-specific assembly:
`llc -o OUT.s IN.ll`
- Create binary from architecture specific-assembly:
`cc -o OUT IN.s`
- Get more help:
`<TOOL> --help`

- General language reference manual:
<http://llvm.org/docs/LangRef.html>
- Doxygen code documentation:
(well accessible via Google/Bing/DuckDuckGo/...)
<http://llvm.org/doxygen/index.html>
- Full command line tools guide:
<http://llvm.org/docs/CommandGuide/>

Sea of Nodes

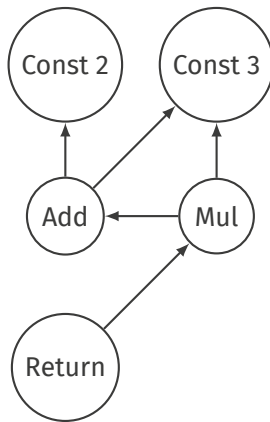
Why Graph-Based IRs?

- Traditional IRs (e.g., SSA, three-address code) use linear instruction lists.
- Optimizations often require complex data/control flow analysis.
- Graph-based IRs encode both data and control dependencies in a single graph.
- Enables more natural and efficient optimizations.

What is a Sea-of-Nodes IR?

- Developed for compilers like Java HotSpot and V8 (JavaScript).
- A unified graph representation of a program.
- Nodes represent operations (e.g., add, load, branch).
- Edges represent:
 - Data dependencies
 - Control dependencies
 - Memory dependencies

Visual Example (Simple Sea-of-Nodes Graph)



Example: `return (2 + 3) * 3`

Pros/Cons

- + Global view of program structure
- + Some optimizations (value numbering, constant folding, ...) simpler/more efficient to implement
- + On-the-fly optimizations
- + Better scheduling and instruction selection
- + Easier construction
- More complex construction traversal
- Steep learning curve
- Explicit side effects
- + Explicit side effects
- Harder to debug
- Requires effective scheduling to produce efficient code
- + Modern compilers perform code motion anyway

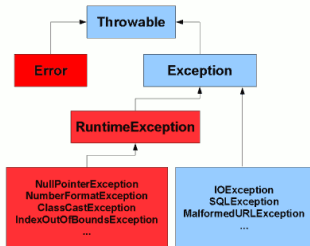
Continuations

Continuation-Passing Style (CPS)

- **continuation-passing style (CPS)**: style of programming in which control is passed **explicitly** in the form of a **continuation**
- **direct style**: usual style of programming
- **continuation**: Function that does **not** return: $A \rightarrow \perp$.
- $\text{int} \rightarrow \text{bool} \Rightarrow \{\text{int}, \text{bool} \rightarrow \perp\} \rightarrow \perp$

<pre>(define (fac n) (if (= n 0) 1 (* n (fac (- n 1)))))</pre>	<pre>(define (fac& n k) (=& n 0 (lambda (b) (if b (k 1) (-& n 1 (lambda (nm1) (fac& nm1 (lambda (f) (*& n f k))))))))))</pre>
--	---

Java: Checked Exceptions as Continuations



```
File file = new File("foo.txt");
try {
    // FileInputStream(String name) throws FileNotFoundException
    var stream = new FileInputStream(file);
    foo(stream)
} catch (FileNotFoundException e) {
    e.printStackTrace();
}
// k
```

```
mkFileInputStream = λx: {file: File, throw: FileNotFoundException -> ⊥, return: FileInputStream -> ⊥}.
    if (* file not found *) then throw (mkFileNotFoundException (...*))
    else return (mkFileNotFound (...*))
```

```
mkFileInputStream {file=File, throw=λe: FileNotFoundException. printStackTrace (e k)
    return=λstream: FileInputStream. foo (stream k)}
```

```

int f(bool b) {
  int x;
  if (b) {
    x = 23;
  } else {
    x = 42;
  }
  return x;
}

```

```

define i32 @f(i1 %b) {
entry:
    br i1 %b, label %t, label %f
t:
    br label %n
f:
    br label %n
n:
    %p = phi i32 [23, %t], [42, %f]
    ret i32 %p
}

```

```

f = λ x:{b: i1, ret: i32 -> ⊥}.
    if x#b then t () else f () where
t = λ _:{}.
    n 23
f = λ _:{}.
    n 42
n = λ p: i32.
    x#ret p

```

MimIR

Domain Specific Languages Are Everywhere!



Domain Specific Languages Are Everywhere!



Domain Specific Languages Are Everywhere!



Domain Specific Languages Are Everywhere!



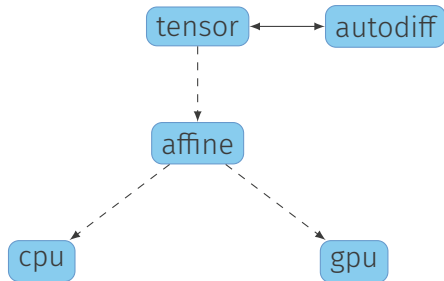
Optimization Beyond Library Interfaces

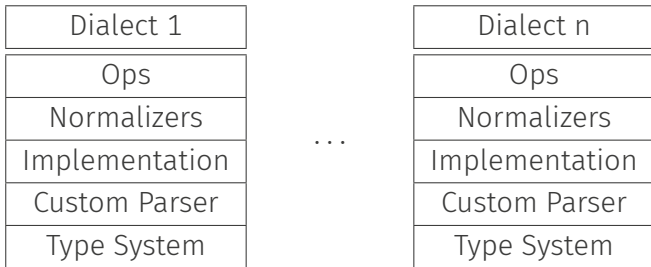
$$M \cdot 0 = 0$$

$$M \cdot I = M$$

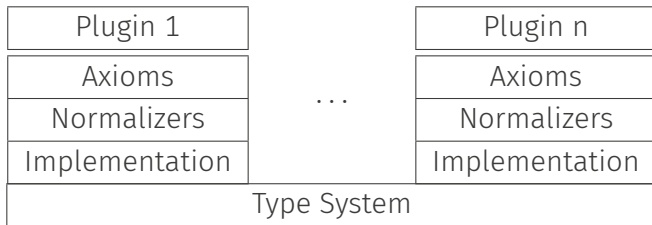
$$\underbrace{A^{10 \times 30} \cdot (B^{30 \times 5} \cdot C^{5 \times 60})}_{27,000 \text{ operations}} = \underbrace{(A^{10 \times 30} \cdot B^{30 \times 5}) \cdot C^{5 \times 60}}_{4,500 \text{ operations}}$$

DSLs: Interaction & Reusability





- Common infrastructure for compiler projects
- Vast impact in the compiler community
- MLIR does not have a semantics per se – by design!
- Only limited support for higher-order functions
- No (higher-order) polymorphism, no dependent types



- Compiler IR: “Sea-of-nodes” program graph
- Typed λ -calculus (CC, PTS)
- Textual interface: Mim

tensor.mim

```
let %tensor.Ring = [  
  T: *,           // element type  
  _0: T,          // zero element  
  _1: T,          // one element  
  add: [T, T] → T,  
  mul: [T, T] → T,  
];  
  
axm %tensor.matmul: [R: %tensor.Ring]  
  → {l m n: Nat}  
  → [«l, m; R#T», «m, n; R#T»]  
  → «l, n; R#T»;
```

- Normalizations

$$M \cdot 0 \triangleright 0$$

$$M \cdot I \triangleright M$$

...

- Other optimizations:
 - Re-associate matrix chain multiplication
 - Lowering passes
 - ...

Using a Plugin

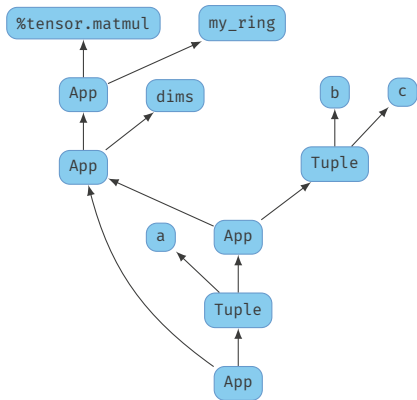
test_tensor.mim

```
plugin tensor;
```

// ...

```
let mul = %tensor.matmul my_ring;
```

```
let res = mul (a, mul (b, c));
```



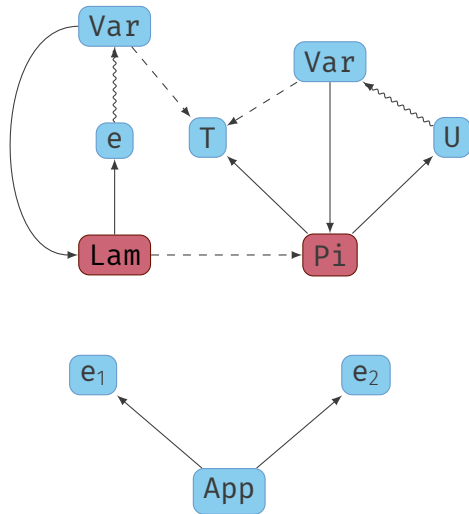
In addition: Every node has a type edge.

MimIR: Functions

Lambda: **lam** $f(x: T): U = e$

Pi: $[x: T] \rightarrow U$

Application: $e_1 \ e_2$



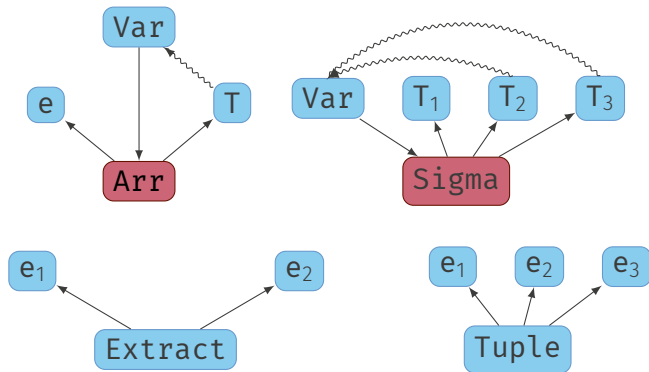
MimIR: Tuples & Arrays

Sigma: $[x_1: T_1, x_2: T_2, T_3]$

Array: $\langle x: e; T \rangle$

Tuple: (e_1, e_2, e_3)

Extract: $e_1 \# e_2$



Extraction from n -tuple/array only allowed with index of type **Idx** n :

```
let a = (x, y, z)#03;  
let b = (x, y, z)#13;  
let c = (x, y, z)#23;
```

```
let arr: «n; Nat» = /*...*/;  
let i: Idx n      = /*...*/;  
let elem: Nat     = arr#i;
```

Smart constructors: All expressions are normalized **during** construction!

Smart constructors: All expressions are normalized **during** construction!

$(0, 1, 2, 3)\#2_4 \triangleright 2$

Smart constructors: All expressions are normalized **during** construction!

`(0, 1, 2, 3)#24` ▷ 2

`[Nat, Nat, Nat]` ▷ «3; Nat»

Smart constructors: All expressions are normalized **during** construction!

$(0, 1, 2, 3)\#2_4 \triangleright 2$

$[\text{Nat}, \text{Nat}, \text{Nat}] \triangleright \llbracket 3; \text{Nat} \rrbracket$

$\llbracket i: 3; F\ i \rrbracket \triangleright [F\ 0_3, F\ 1_3, F\ 2_3]$

Smart constructors: All expressions are normalized **during** construction!

$(0, 1, 2, 3)\#2_4 \triangleright 2$

$[\text{Nat}, \text{Nat}, \text{Nat}] \triangleright \llbracket 3; \text{Nat} \rrbracket$

$\llbracket i: 3; F\ i \rrbracket \triangleright [F\ 0_3, F\ 1_3, F\ 2_3]$

$[T] \triangleright T$

Smart constructors: All expressions are normalized **during** construction!

$(0, 1, 2, 3)\#2_4 \triangleright 2$

$[\text{Nat}, \text{Nat}, \text{Nat}] \triangleright \llbracket 3; \text{Nat} \rrbracket$

$\llbracket i: 3; F\ i \rrbracket \triangleright [F\ 0_3, F\ 1_3, F\ 2_3]$

$[T] \triangleright T$

$(e) \triangleright e$

Smart constructors: All expressions are normalized **during** construction!

$(0, 1, 2, 3)\#2_4 \triangleright 2$

$[\text{Nat}, \text{Nat}, \text{Nat}] \triangleright \llbracket 3; \text{Nat} \rrbracket$

$\llbracket i: 3; F\ i \rrbracket \triangleright [F\ 0_3, F\ 1_3, F\ 2_3]$

$[T] \triangleright T$

$(e) \triangleright e$

$e\#0_1 \triangleright e$

Normalization: Plugins

Plugins can introduce their own normalizations:

Normalization: Plugins

Plugins can introduce their own normalizations:

%core.nat.add (3, 4) ▷ 7

Normalization: Plugins

Plugins can introduce their own normalizations:

%core.nat.add (3, 4) ▷ 7

%core.nat.add (x, 0) ▷ x

Plugins can introduce their own normalizations:

`%core.nat.add` (3, 4) ▷ 7

`%core.nat.add` (x, 0) ▷ x

`%tensor.matmul` (m, id) ▷ m

Type Checking

- Normalizations not only low-hanging fruits for optimizations but also important for type-checking:
 - Type expressions are normalized.
 - $T((n + 0) + x)$ vs. $T(x + (0 + n))$
- Dependent type checking with full recursive functions undecidable!
- MimIR: filter-based (partial) evaluation to decide type checking:

```
lam pow(a b: Nat)@%core.pe.known b: Nat =  
  (%core.nat.mul (a, pow(a, %core.nat.sub (b, 1))), 1)#(%core.ncmp.e (b, 0));  
  
lam f(n: Nat, x: «%core.nat.mul (n, %core.nat.mul (n, n)); Nat»): R = /* ... */;  
  
let y: «pow (n, 3); Nat» = /*...*/;  
let res = f (n, y);
```

- Progress & Preservation: Rocq proof for the core calculus
- Custom axioms/normalizers can violate
 - preservation:
 - $3 + 5 \triangleright ()$
 - progress:
 - `%core.bitcast (Idx 3) 57`
 - `%mem.load`ing dangling pointer
 - ...

Continuations

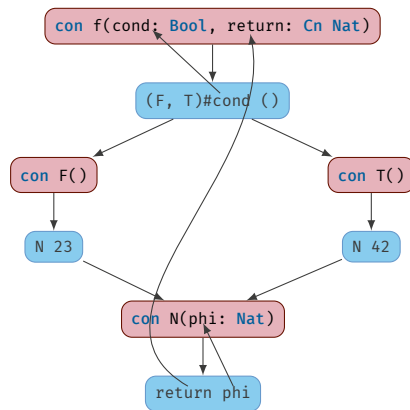
- Continuations are functions that don't return:

$\text{Cn } T \quad := \quad T \rightarrow \perp$

$\text{con } f(x: T) = e \quad := \quad \text{lam } f(x: T): \perp = e$

- Allows for unstructured control-flow
- Similar to CFG + SSA

```
con f(cond: Bool, ret: Cn Nat) =  
  (F,T)#cond () where  
    con F() = N 23;  
    con T() = N 42;  
    con N(phi: Nat) = ret phi;  
end;
```

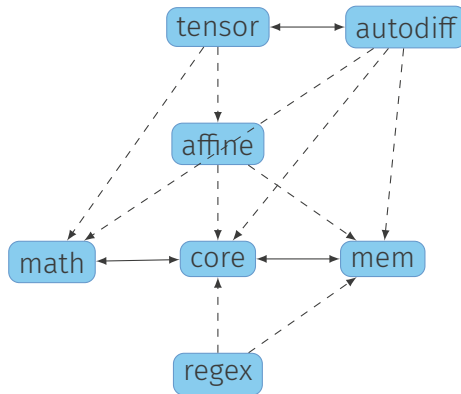


Many optimization can be expressed as **specializations**!

```
con iter {A: *} (n: Nat) (body: Cn [Cn A][A]) (exit: Cn A) (acc: A)@tt =  
  head(0, acc) where  
    con head (i: Nat, acc: A)@%core.pe.known n =  
      // ...  
    end;  
  
iter 12 body exit init
```

Goals

- High-performance
- Expressive enough to host variety of DSLs
- Enables interaction & reusability



Conclusion

tensor.mim

```
let %tensor.Ring = [  
  T: *,           // element type  
  _0: T,          // zero element  
  _1: T,          // one element  
  add: [T, T] → T,  
  mul: [T, T] → T,  
];  
  
axm %tensor.matmul: [R: %tensor.Ring]  
  → {l m n: Nat}  
  → [«l, m; R#T», «m, n; R#T»]  
  → «l, n; R#T»;
```

- Normalizations

$$M \cdot 0 \triangleright 0$$

$$M \cdot I \triangleright M$$

...

- Other optimizations:
 - Re-associate matrix chain multiplication
 - Lowering passes
 - ...

<https://anydsl.github.io/MimIR/>